

Week-5 Assignment

Q1: What are the key limitations of traditional MapReduce that make **Spark** a preferred choice for modern big data processing?

Ans:

- **Disk VS in-memory processing:**

As traditional map reduce stores intermediate results and write it over HDFS which increases latency and longer time to process whereas Spark retain data in RAM avoiding disk writes and I/O bottlenecks.

- **Batch VS real time processing**

Map reduce provides batch processing and do not support real time processing whereas Spark supports batch processing as well as real time processing.

- **Inefficient for iterative workloads:**

Machine learning and graph algorithms require repeated computation. MapReduce perform I/O after iteration whereas spark uses in-memory computation resulting increased performance.

- **Complexity:**

Map reduce uses complex queries whereas spark provides easy to use high level APIs for Java, Python, Scala and R.

- **Fault tolerance:**

Although both the systems are fault tolerant but Spark uses Resilient Distributed datasets (RDD) to reduce recomputation overhead.

Q2: Explain how Spark uses **In-Memory Computing** to speed up iterative machine learning algorithms compared to disk-based systems.

Ans:

- Spark uses In-Memory Computing for iterative machine learning algorithms.
- Algorithms like logistic regression, K-means iterates over same data. Map reduce read and write over disk after every iteration which increases I/O overhead and execution time.

- But spark retains intermediate data in RAM using Resilient Distributed Datasets (RDDs) and caches frequently accessed data. Therefore, speeding up iterative machine algorithms and improves performance.

Q3: Write a code snippet to remove all **duplicate rows** from a DataFrame based on a specific set of columns: user_id and transaction_date.

Ans: Code Snippet-

```
df=df.dropDuplicates(subset=['user_id','transaction_date'])
```

We use subset parameter inside dropDuplicates() function to remove duplicate rows based only on specific columns i.e user_id and transaction_date.

Q4: Given a DataFrame df_sales, write a query to **filter** for rows where the region is 'West' and then **group by** product_category to find the average sale_amount.

```
Ans: df_filtered=df.filter(col('region')==west')
```

```
Result=df_filtered.groupby("product_catagory").avg("sale_amount")
```

Q5: What is the difference between .na.drop() and .na.fill()? Provide a code example of filling null values in a status column with the string 'Unknown'.

Ans: .na.drop() :-It is used to remove rows from a dataframe if it contains null values.

.na.fill():-Pyspark command to replace missing values with specified values.

```
df=df.na.fill("Unknown", subset=["status"])
```

Q6: Write a query to find the **total count** of records for each city in a DataFrame, but only for cities where the count is greater than 100.

```
Ans: result=df.groupby("city").agg(count("city").alias("count")).filter(col("count")>100)
```

Q7: How does the **immutability** of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Ans: DataFrames in Spark are immutable that is you can not change or modify the existing dataframe. Therefore data cleaning operations like drop(), fill() , withColumnRenamed() etc. create and returns a new Dataframe instead of altering the original one. This immutability ensures fault tolerance and help in parallel processing.

Q8: Write a Spark command to **filter** a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Ans: `df=df.filter((col("age")>=18) & (col("age") <=30) & (col("subscription")=="Premium"))`

Q9: When **cleaning a dataset**, why is it often better to handle null values before performing mathematical aggregations like `sum()` or `avg()`?

Ans: It is often better to handle null values before performing aggregations because null values or missing values can affect the expected outcomes. For ex: you want to find avg of students and out of 10, 4 values are missing and then the `avg()` function will not provide result for entire population that can lead to incomplete or misleading result and affecting subsequent analysis.

Q10: Write the code to **revise** a column named `raw_timestamp` by casting it to a `TimestampType` and renaming it to `event_time`.

```
df_renamed=df.withColumn(  
    "raw_timestamp", col("raw_timestamp").cast("timestamp")  
)  
.withColumnRenamed("raw_timestamp", "event_time")
```

Q11: Explain the "Shuffle" process that occurs during a **grouping** operation. Why is it considered a wide transformation?

Ans: Spark redistributes data with same key to the same partition, this process is known as shuffling. It involves moving data across partitions which increases I/O and execution time. It is considered as wide transformation because output partition depends on multiple input partitions unlike narrow transformations which depend on single input partition.

For Ex:

Suppose you have

Partition 1:

(A, 10)

(B, 5)

(A, 30)

Partition 2:

(A, 20)

(B, 15)

(C, 25)

During shuffling, same keys goes to same partition

Partition 1:

(A, 10)

(A, 20)

(A, 30)

Partition 2:

(B, 5)

(B, 15)

(C, 25)

Q12: Write a code snippet that identifies and **removes rows** where the email column contains null values OR the username is an empty string.

Ans:

```
df_cleaned=df.filter(  
col("email").isNull() & (col("username")!= "")  
)
```

Q13: How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Ans: The .agg() function can apply multiple aggregate functions to a column in a single operation. For example, to calculate the minimum, maximum, and average values of the price column:

```
result = df.agg(  
    min("price").alias("min_price"),  
    max("price").alias("max_price"),  
    avg("price").alias("mean_price")  
)
```

This computes the minimum, maximum, and mean of the price column simultaneously.

Q14: In the context of **cleaning a dataset**, what is the risk of using inferSchema=true when your source data contains messy or inconsistent date formats?

Ans: When inferSchema=True is used, Spark automatically determines the data types based on the input data. If the source data contains messy or inconsistent date formats, Spark may infer an incorrect data type or interpret the same column differently. This can lead to parsing errors, null values, incorrect results, and issues in subsequent analysis. Therefore, it is often better to explicitly define the schema for such datasets.

Q15: Write a final processing pipeline that:

1. Filters out **duplicates**.
2. Fills **null** prices with 0.
3. Groups by store_id to calculate total revenue.

Ans: from pyspark.sql.functions import sum

```
result = (  
    df.dropDuplicates()  
    .na.fill(0, subset=["price"])  
    .groupBy("store_id")  
    .agg(sum("price").alias("total_revenue"))  
)
```

This pipeline:

1. Removes duplicate rows using dropDuplicates().
2. Replaces null values in the price column with 0.
3. Groups records by store_id and calculates the total revenue for each store.